# 拓展功能集

## 从零手搓 Step5 教程

2026 FPGA 创新设计赛道（安路）赛题二

视觉方向 • Step5_expansion

读者：零基础队员（建议先读完 Step1～4 教程）
目标：把评分表的进阶/扩展项逐个补齐——多算法、运动检测、画质调节、声光
前置要求：完成 Step1～4 的手搓或至少精读
预计学习时间：10～14 天（每天 2～3 小时）

**本教程与你手上的代码是什么关系**

Step1～4 打通了”看 - 认 - 动 - 显”；Step5 是**加分项集中营**——
官方边缘检测核接入、帧差运动、亮度对比度、三行 OSD、声光反馈。

# 目录

**第一部分　从零学习教程** **3**

**1　你在造什么** **3**
1.1　一句话 . . . 3
1.2　五件套总览 . . . 3
1.3　做完是什么效果 . . . 3

**2　阶段 0：官方例程当积木（1 天）** **3**
2.1　edge_detect_96 是什么 . . . 3
2.2　接入姿势：并行分支，不是替换 . . . 4
2.3　练习 . . . 4

**3　阶段 1：帧边界切换的艺术（2 天）** **4**
3.1　为什么帧中间切会出事故 . . . 4
3.2　解法：帧首拍锁存 . . . 4
3.3　练习 . . . 4

**4　阶段 2：亮度与对比度（1～2 天）** **4**
4.1　一条公式 . . . 5
4.2　两个设计决定 . . . 5
4.3　练习 . . . 5

**5　阶段 3：帧差运动检测（3 天）** **5**
5.1　原理：一比特的” 不同” . . . 5
5.2　代价：一帧的” 记忆” . . . 5
5.3　亚门限也要” 报告” . . . 6
5.4　练习 . . . 6

**6　阶段 4：仿真与综合的” 不一致” 陷阱（2 天）** **6**
6.1　陷阱 1：越界写，仿真沉默、硬件回绕 . . . 6
6.2　陷阱 2：首帧门控靠 X ” 碰巧” 工作 . . . 6
6.3　练习 . . . 6

**7　阶段 5：三行 OSD 与声光（1～2 天）** **6**
7.1　OSD 布局 . . . 6
7.2　声光反馈 . . . 7
7.3　练习 . . . 7

**8　阶段 6：跑通回归（1 天）** **7**
8.1　16 项一键回归 . . . 7

**第二部分　工程实现详解** **8**

**9　工程总览** **8**
9.1　文件清单 . . . . . 8
9.2　目标包 v1.0 到 v1.1.0 . . . . . 8

**10 模块详解：edge_detect_96（官方核）** **8**
10.1 粒度对齐：4 像素进、4 像素出 . . . . . 8

**11 模块详解：video_adj** **9**
11.1 Q6 公式的位精确实现 . . . . . 9
11.2 挂在哪条路上 . . . . . 9

**12 模块详解：motion_detect v2** **9**
12.1 存储与地址 . . . . . 9
12.2 v2 修正的两处（都有实战故事） . . . . . 10
12.3 接口语义（写进 IF-VISION） . . . . . 10

**13 既有模块的改造** **10**
13.1 detect_top：+fid +motion 挂载 . . . . . 10
13.2 osd_track v2：三行文本与双框 . . . . . 10
13.3 vision_top v2：五合一顶层 . . . . . 10

**14 测试平台设计与回归** **11**
14.1 tb_motion_detect：5 帧状态机全覆盖 . . . . . 11
14.2 回归现状：18 项全绿（本册时点为 16 项 + 一处待办，该待办已补齐） . . . . . 11
14.3 Step5 迭代复盘（7 轮） . . . . . 11

**15 上板挂载说明（给上板组）** **12**

**16 小结：Step5 的可复用产出** **12**

**结语** **12**

**总结与下一步** **13**
Step5 学到了什么 . . . . . 13
下一步做什么 . . . . . 13

# 从零学习教程

## 1 你在造什么

### 1.1 一句话

**对着评分表编程：基础项做完之后，每一个进阶/扩展条目都是一个 独立小功能，这一步把它们一次做齐。**

### 1.2 五件套总览

| 功能 | 对应要求 | 实现 | 用户看到 |
|---|---|---|---|
| 多算法切换 | 进阶（1） | `edge_detect_96` + 帧首 mux | 按键切边缘图 |
| 延迟可视化 | 进阶（3） | fid 随包传递 | OSD 的 D 字段 |
| 运动检测 | 进阶（4） | `motion_detect` | 橙色包围框 |
| 三行 OSD+ 跑马灯 | 扩展（1） | `osd_track` v2/v3 | 文字层 |
| 亮度对比度 | 扩展（4） | `video_adj` | 实时调节 |
| 声光反馈 | 演示加分 | `led_buzzer` | 绿灯/蜂鸣 |

### 1.3 做完是什么效果

按一下按键，画面从” 彩色跟踪” 切换成” 黑白边缘图”（无缝，无撕裂）；有人在镜头前挥手，目标框外多出一个**橙色**运动包围框；拧亮度旋钮，画面明暗实时变化且左上角 B/C 字段跟着变；锁定目标时蜂鸣器短鸣一声、绿灯常亮。

**学完 Step5 你能做到**

- 把官方例程核” 零适配” 接进自己的链路
- 实现帧边界无缝的算法切换并解释为什么不能帧中切
- 写 Q6 定点的亮度/对比度流水线
- 用 1bit/像素的块存储器做帧差运动检测
- 说出两处” 仿真过了上板翻车” 的经典陷阱及防范

## 2 阶段 0：官方例程当积木（1 天）

### 2.1 edge_detect_96 是什么

官方 Lab2 给了一个 Sobel 3x3 边缘检测核（`edge_detect_96.v`），工作在 96bit AXI-Stream 上（一次 4 像素），输出边缘点白色、非边缘黑色。**端口和官方视频链路完全兼容：**tuser/tlast/tready 直通，挂上就能用。

### 2.2 接入姿势：并行分支，不是替换

```
96bit 入 ----+---- 直通（显示主线不动）
             +---- detect_top（颜色检测分支，常开）
             +---- edge_detect_96（边缘分支，常开）
                        |
             mux <- algo_q（帧首锁存的选择信号）
```

两个算法分支**同时运行**，切换只动 mux——不是”停一个启一个”。这样切换零启动延迟，也避免了”新算法第一帧还没出图”的尴尬。

**读官方代码的三个动作**

1）读端口和参数（能配置什么）；2）读时序（延迟几拍、tuser/tlast 怎么传递）；3）读粒度（它按 4 像素/拍工作，输出 2 个判决再复制成 4 个——了解这点才能对上拍）。本工程把它当**黑盒积木**用，零修改。

### 2.3 练习

1. 画出三分支（直通/检测/边缘）的结构图，标出 mux 位置
2. 思考：边缘分支和检测分支各占多少逻辑资源？为什么要”常开”而不是按需启动？

## 3 阶段 1：帧边界切换的艺术（2 天）

### 3.1 为什么帧中间切会出事故

mux 前面是两条**延迟不同的流水线**：检测链约 9 拍 + 平滑行缓存，边缘核约 5 拍。如果在帧中间拨动选择信号：

```
帧中第 300 拍切换：
  第 0~299 像素：来自 A 链（当前帧第 0~299 像素）
  第 300~ 像素：  来自 B 链（同一帧，但 B 链进度不同 -> 时间错位）
  -> 画面左右两半来自" 不同时刻"，出现垂直错缝
```

### 3.2 解法：帧首拍锁存

```
if (I_tvalid && I_tuser)      // 只在每帧第一拍采样按键
    algo_q <= I_algo_edge;
// 帧内 mux 恒定 -> 切换永远发生在帧与帧之间
```

切换延迟最多 1 帧（16.7ms）——人眼无感，画面无撕裂。

### 3.3 练习

1. 算一算：两条链延迟差 4 拍时，帧中切换的垂直错缝是几个像素？
2. 思考：为什么采样条件是 `I_tvalid && I_tuser` 而不是单独 tuser？

## 4 阶段 2：亮度与对比度（1～2 天）

### 4.1 一条公式

$$out = clamp\big(((c - 128) \times C \gg 6) + 128 + B\big)$$

三个动作：**居中**（减 128，让” 不增不减” 的乘法不动中灰）、**缩放**（C 是 Q6 定点，64 表示 1.0 倍）、**平移**（B 有符号加）。最后 clamp 到 0..255。

```
v = c - 128;                    // 居中
v = (v * C) >>> 6;              // Q6: 64=1.0 倍, 192=3.0 倍
v = v + 128 + B;                // 回中 + 亮度
clamp 到 [0,255]                // 溢出保护
```

### 4.2 两个设计决定

**决定 1：只放显示路径**

调节挂在 OSD 之前的显示通路上，**检测分支吃的是原图**——你把画面调到全黑，跟踪照样工作（检测不受显示参数影响）。这也是评分表” 实时调节**显示参数**” 的本意。

**决定 2：单通道函数复用**

RGB 三通道完全同构，RTL 写一个 `adj_ch` 函数调用三次——和 Python 金标准一个函数三处调用的结构完全对应，验证时逐通道等价。

### 4.3 练习

1. 手算：c=100, C=128（2.0 倍）, B=16 时的输出
2. 手算：c=128 时任何 C/B 组合下输出是多少？为什么？
3. 思考：为什么 C 用 Q6（64=1.0）而不是直接存小数？

## 5 阶段 3：帧差运动检测（3 天）

### 5.1 原理：一比特的” 不同”

运动 = 这帧和上帧不一样。对**二值掩码**（Step2 平滑后的 0/1 图）做帧差：

```
diff = cur_mask ^ prev_mask     // 异或: 不一样的地方为 1
cnt = sum(diff)                 // 变化像素总数
bbox = diff 的包围框            // 变化发生在哪里
valid = (cnt >= MOT_TH)         // 变化够多才算" 有运动"
```

### 5.2 代价：一帧的” 记忆”

需要存上一帧完整掩码：1280x720x1bit = 115KB——块 RAM（ERAM）刚好装下。核心结构是**同地址读旧写新**：

```
addr = row_base + x;            // 当前行列
old  = prev_mem[addr];          // 先读: 上帧的值
prev_mem[addr] <= I_mask;       // 后写: 本帧的值盖进去
```

和 Step2 平滑的索引行缓存同思想——读写同指针，天然对齐。

### 5.3 亚门限也要”报告”

每帧末锁存 cnt/bbox 并输出 1 拍选通脉冲。**变化少于门限的帧照样输出**（valid=0）——下游靠 valid 区分”没运动”和”这帧没算”，可以放心清掉运动指示，不会出现”橙色框赖着不走”。

### 5.4 练习

1. 手算：目标从 (10,10) 挪到 (30,20)，8x8 块，diff 像素数是多少？（提示：旧位消失 + 新位出现，两块不重叠）
2. 思考：为什么对”掩码”做帧差而不是对原始 RGB？（提示：存储量）
3. 估算：如果存 RGB 全帧要做运动检测，需要多少块 RAM？

## 6 阶段 4：仿真与综合的”不一致”陷阱（2 天）

这是 Step5 最有价值的部分：**两个仿真里完全看不出、上板才爆的坑**，都在运动检测模块里真实抓到过。

### 6.1 陷阱 1：越界写，仿真沉默、硬件回绕

帧尾滚动地址 `row_base += W` 后，下一帧 sof 拍若用**遗留** row_base 算写地址，会越界：

- **仿真**：越界写被仿真器**沉默忽略**——什么都不发生，看不出错误；
- **综合**：地址位宽截断，**回绕**写进 0 号单元——把 (0,0) 的前帧参考改写，行为悄悄不同。

**铁律一：帧首拍的地址必须显式覆盖**

sof 拍像素就是 (0,0)，把写地址**强制为 0**（语义本来就正确），绝不信赖上一帧滚出来的遗留值。

### 6.2 陷阱 2：首帧门控靠 X ”碰巧”工作

首帧还没有”上一帧”可比。初版用 `started` 标志（首帧 sof 拍置位）：

- **仿真**：RAM 未初始化读出 X，`mask ^ X = X`，判决恒假——**碰巧**没报运动；
- **综合**：RAM 初值默认 0，首帧 diff = mask^0 = mask，**整帧被误判成运动**，橙色框满天飞。

**铁律二：门控绝不能依赖 X 传播**

改用 `prev_ready`（**首帧末行**才置位），首帧不比对不输出。X 在仿真里是”薛定谔的守卫”——综合一落地就现原形。

### 6.3 练习

1. 解释为什么”仿真过、上板错”的 bug 比双边都错的更危险
2. 排查你写过的模块：有没有”靠 X 挡”或”靠遗留值”的地方？

## 7 阶段 5：三行 OSD 与声光（1～2 天）

### 7.1 OSD 布局

```
y=16   TRK0 LOCK X0640Y0360      L1: 算法/组号/锁定态/坐标
y=40   D03 B+16 C08 M1839        L2: 延迟/亮度/对比度/运动计数
y=64   2026 FPGA CONTEST - ANLOGIC ...  L3: 跑马灯 (青色)
画面:  绿准星 + 红目标框 + 橙运动框
```

**优先级：L1 > L2 > L3 > 准星 > 目标框 > 运动框 > 底图**

L1 前缀随算法变（TRK/EDG），L2 的 D 字段用” 显示帧号减包内采集帧号” 实时算出链路延迟，M 字段直接来自运动检测的 cnt。

### 7.2 声光反馈

```
锁定： 绿灯常亮
丢失： 红灯 1Hz 闪
锁定沿：蜂鸣 200ms（从" 丢" 变" 锁" 的那一拍触发）
```

GPIO 直驱，无任何协议——演示时” 听见哔一声” 就是最好的锁定提示。

### 7.3 练习

1. 画出 L1/L2/L3 的像素区域（x 范围、y 范围）
2. 思考：为什么” 锁定沿” 触发蜂鸣而不是” 锁定电平”？

## 8 阶段 6：跑通回归（1 天）

### 8.1 16 项一键回归

```
cd D:\fpga-ptz-tracker\tools
python run_all.py          # 全量 16 个测试
```

Step5 新增 `tb_motion_detect`（第 16 项），场景设计覆盖全部状态转移：

| 帧 | 场景 | 期望 |
|---|---|---|
| f0 | 静止背景 | 只填缓存, 无输出 |
| f1 | 出现 8x8 块 | cnt=64, bbox(10,10)-(17,17) |
| f2 | 块移动 | cnt=128（旧位消失 + 新位出现） |
| f3 | 块消失 | cnt=64（原位置残影） |
| f4 | 5 个孤立像素 | cnt=5, valid=0（**亚门限拒报**） |

**Step5 学习清单（全部打勾就过关）**

- 能说清” 并行分支 + 帧首 mux” 的切换机制
- 能手算 Q6 亮度对比度的一像素变换
- 能解释帧差检测的结构（XOR/累加/门限）与亚门限语义
- 能复述两处仿真/综合不一致陷阱及铁律
- 跑通 16 项回归，说清 motion 测试 5 帧各验什么

# 工程实现详解

**这一部分写什么**

Step5 的实现分两类：4 个新 RTL（边缘核接入、画质调节、运动检测、跑马灯字库）+ 3 个既有模块改造（detect_top / osd_track / vision_top），外加 7 轮迭代与两处”仿真过、上板会翻车”的修复。对应目录：`03_vision/step5_expansion/`。

## 9 工程总览

### 9.1 文件清单

| 文件 | 类型 | 行数 | 职责 |
|---|---|---|---|
| `edge_detect_96.v` | 官方 | 149 | Sobel 边缘核（Lab2，零修改复用） |
| `video_adj.v` | RTL | 47 | 亮度/对比度（1 拍流水） |
| `motion_detect.v` | RTL | 117 | 帧差运动检测 v2（修正版） |
| `marquee_rom_64.v` | 生成 | 1020 | 跑马灯 63 字符字库（脚本生成） |
| `detect_top.v` | 改造 | — | +fid 计数器 +motion 分支挂载 |
| `osd_track.v` | 改造 | — | v2 三行文本 + 双框（后升级 v3 双缓冲） |
| `vision_top.v` | 改造 | — | v2 算法 mux + video_adj + led_buzzer |
| `tb/tb_motion_detect.v` | 测试 | 117 | 5 帧场景逐字段比对 |

### 9.2 目标包 v1.0 到 v1.1.0

包从 68bit 扩到 **135bit**（IF-VISION v1.1.0），新增字段：algo(1) + fid(8) + mot_valid(1) + mot_cnt(14) + mot bbox(62)。老字段位置不动、新字段插在高位——osd_track 解包代码按位段逐一对齐，保留位 bit0 强制补 0（Step4 事故的沉淀）。

## 10 模块详解：edge_detect_96（官方核）

### 10.1 粒度对齐：4 像素进、4 像素出

官方核按 96bit 流的原生粒度工作：每拍 4 像素，Sobel 在灰度上算出 2 个边缘判决，**复制**回 4 像素输出。这套粒度与我们整条链路一致，所以挂上即用：

```
edge_detect_96 #(.EDGE_TH(80)) u_edge (...);   // 官方默认阈值
// tuser/tlast/tready 直通，输出再接一个 ser96to24 降带宽
```

**复用官方代码的三条纪律**

1）**不改**（改动会破坏与官方链路的兼容性，出问题没参照）；
2）**包一层**（阈值等可调参数从顶层传参，不在官方文件里改默认值）；
3）**单独验证**（官方核在 Lab2 已验证，我们验证的是”接线正确”——tb_vision_top 的整帧比对顺带覆盖了边缘分支的 mux 路径）。

## 11 模块详解：video_adj

47 行的小模块，三处实现决定值得学：

### 11.1 Q6 公式的位精确实现

```
function [7:0] adj_ch(input [7:0] c, input [6:0] bright, input [7:0] contrast);
    reg signed [16:0] v;
    begin
        v = $signed({1'b0, c}) - 17'sd128;          // 居中 (有符号!)
        v = (v * $signed({1'b0, contrast})) >>> 6; // Q6 缩放 (算术右移)
        v = v + 17'sd128 + $signed({1'b0, bright});// 回中 + 亮度
        if (v < 0)          adj_ch = 8'd0;
        else if (v > 255)   adj_ch = 8'd255;
        else                adj_ch = v[7:0];
    end
endfunction
```

`c - 128` 必须有符号（暗部像素是负偏移），乘法后 `>>>6` 与 Python 金标准的 floor 移位逐位一致——Step2 沉淀的”有符号纪律”第三次落地。

### 11.2 挂在哪条路上

```
mux(algo_q) -> video_adj -> osd_track -> 显示
                  ^
      检测分支吃的是 mux 之前的原图，不受调节影响
```

注意顺序：调节在 **OSD 之前**——OSD 文字不受亮度调节影响（调到全黑时状态行依然清晰可读，演示不会”自残”）。

## 12 模块详解：motion_detect v2

### 12.1 存储与地址

```
(* ram_style = "block" *) reg prev_mem [0:TOTAL-1];  // 1bit/像素, 115KB
wire [20:0] addr = I_sof ? 21'd0 : (row_base + {10'd0, x});
wire        old  = prev_mem[addr];
wire        diff = I_mask ^ old;
// 同地址读旧写新 (与 Step2 平滑行缓存同思想)
```

### 12.2 v2 修正的两处（都有实战故事）

**修正 1：sof 拍写地址强制 0**

帧尾 `row_base += W` 滚动后，下一帧 sof 拍曾用遗留 row_base：**仿真**越界写沉默忽略（(0,0) 参考停在旧值）；**综合** ERAM 地址回绕写 0 号单元。同一个 RTL，两种”物性”。修复：sof 拍地址强制 0——该拍像素恰是 (0,0)，语义本来就正确。

**修正 2：prev_ready 门控**

初版 `started` 在首帧 sof 拍置位，挡不住首帧后续像素：仿真靠”RAM 读 X 使 diff 恒假” **碰巧**不报；综合 RAM 初值 0 会把首帧误判为全帧运动。修复：改 `prev_ready`（首帧末行置位），首帧不比对不输出。**门控靠显式状态位，绝不靠 X。**

### 12.3 接口语义（写进 IF-VISION）

每帧末锁存结果并输出 1 拍的 `O_mot_valid` 选通脉冲。**亚门限帧也更新 cnt/bbox 并出选通**，是否算运动由 `valid=(cnt>=MOT_TH)` 表达——下游 OSD 据此可以**清掉**运动指示，不会出现”橙色框赖着不走”。已知约定：(0,0) 像素每帧只写缓存不比对（sof 拍结构，影响 1/921600，金标准同语义）。

## 13 既有模块的改造

### 13.1 detect_top：+fid +motion 挂载

```
// 进阶 (3): 采集帧号，每输入帧 sof 自增
reg [7:0] fid;
always @(posedge I_clk or negedge I_rst_n)
    if (!I_rst_n) fid <= 8'd0;
    else if (I_tvalid && I_tuser) fid <= fid + 8'd1;

// 进阶 (4): 运动分支，与 centroid 共享平滑掩码流
motion_detect #(.MOT_TH(64)) u_motion (.*, .I_mask(m_mask), ...);
```

迭代 1 的教训：motion 块一开始插在 `m_*` wire 声明之前，隐式声明与显式声明冲突编译错——**Verilog 隐式 wire 是事故之源**，新代码统一追加在文件尾部（本项目惯例）。

### 13.2 osd_track v2：三行文本与双框

L1 加算法前缀（TRK/EDG 由包内 algo 位选择），L2 全新遥测行（D/B/C/M 四字段），L3 跑马灯（`marquee_rom_64` 生成字库，63 字符 x 16 行 case 表）。运动框橙色 `FF8000`，优先级插在目标框之后——它信息量最低，被遮挡时最先让路。

### 13.3 vision_top v2：五合一顶层

```
mux(帧首锁存 algo_q) -> video_adj -> osd_track -> OSD'd vpix 出
     |              ^
det 分支 (常开)   edge 分支 (常开)
135bit pack = {...}  -> cdc -> osd_track
```

```
led_buzzer(pix 域)   -> O_led_g/r, O_buzzer
```

迭代 5：vision_top 端口加了 3 个新输入（I_algo_edge/I_bright/ I_contrast），上层 TB 没传——**端口变更必须同步所有实例化点**，run_all 全量回归的价值就是让这类失配 5 分钟内暴露。

## 14 测试平台设计与回归

### 14.1 tb_motion_detect：5 帧状态机全覆盖

64x48 迷你几何（与 centroid 向量同思路：小图跑全逻辑）。期望值由金标准函数 `frame_diff_motion`（位于 `golden_model.py`）逐位生成，打包格式为 15 个十六进制字符（cnt/valid/bbox 六字段移位拼接）。5 帧场景与期望值见 Part I 阶段 6 的表——注意 f4 的**亚门限拒报**（cnt=5，valid=0，但 bbox 照常输出）是单独设计的边界用例。

### 14.2 回归现状：18 项全绿（本册时点为 16 项 + 一处待办，该待办已补齐）

| 状态 | 说明 |
|---|---|
| 18/18 PASS | 含 tb_motion_detect（Step5 新增第 16 项；闭环/算法切换补强后共 18 项） |
| tb_osd_track | 功能断言 + 整帧逐像素比对全过（v2 渲染器补齐后启用） |
| tb_vision_top | 直通/云台断言 + 整帧比对全过（同上） |

**曾经的诚实待办：金标准渲染器 v2（已闭环）**

RTL 的 OSD 已升级到”三行文本 + 跑马灯”，但 Python 端 `render_osd()` 还是 v1（单行无跑马灯）——比对自然失败。工程选择**先禁用帧比对并在 run_all.py 里标注 pending**，而不是让回归长期红着。上板前必须补齐 v2 渲染器并恢复比对；**功能断言（文字内容/坐标/云台方向）目前由 TB 内部断言承担**。这条待办写进了迭代记录与结语，不藏不瞒。**2026-10-03 更新：v2 渲染器已补齐，逐像素比对全面恢复，全量 18/18 全绿。**

### 14.3 Step5 迭代复盘（7 轮）

| 轮 | 现象 | 根因 | 类型 |
|---|---|---|---|
| 1 | detect_top 编译错 | motion 块插在声明前，隐式/显式冲突 | RTL |
| 2 | started 双重驱动 | 底部残留占位行 | RTL |
| 3 | TB 找不到 dut.txt | 重命名 txt 后层次引用未同步 | TB |
| 4 | TB 结构性语法错 | 多轮正则改坏 | TB（备份恢复） |
| 5 | 上层 TB 编译失败 | 顶层加端口 TB 未传 | TB |
| 6 | STEP5_RTL 未定义 | 脚本变量顺序 | 脚本 |
| 7 | 帧比对失败 | 金标准 v1 未跟上 RTL v2 | 流程（已闭环） |

**迭代 4 的处置值得单独学**

TB 被多轮自动修改改坏后，**从备份恢复原始版本，只重新应用**必要的 pack/端口改动——” 改坏了就回到最后已知好点，重放最小 diff”，比在废墟上继续打补丁快一个数量级。备份目录（`fpga-ptz-tracker_backup_*`）不是摆设。

## 15 上板挂载说明（给上板组）

| 按键/输入 | 功能 |
|---|---|
| KEY0..2 / 拨码 | preset_idx 阈值组（0 红 1 绿 2 蓝...） |
| 拨码 | track_en（0 = 云台回中位） |
| 按键 | algo_edge（1 = 边缘显示，帧边界无缝切换） |
| 按键 x2 | 亮度 -/+（对比度固定或再加两键） |
| GPIO | O_led_g / O_led_r / O_buzzer（声光） |

上板三步：

1. 4 个新 RTL + 3 个改造模块加入 TD 工程（edge_detect_96 原样复制，**不许改**）；
2. vision_top 替换原有顶层实例化位置，端口对照 IF-VISION v1.1.0；
3. 先空载跑通显示与 OSD，再接舵机与声光，最后现场标定。

**答辩素材提示**

三行 OSD 是” 评分表自证屏”（Step4 讲过）；运动检测演示建议安排” 静止 3 秒 - 挥手 - 静止” 的节奏，橙色框出现/消失清晰可辨；声光反馈让” 锁定瞬间” 可听——三个感官通道同时确认，演示效果拉满。

## 16 小结：Step5 的可复用产出

- **4 个新模块 + 3 处改造**，进阶（1）（3）（4）与扩展（1）（4）全部落地并有验证；
- **两条上板铁律**：帧首地址显式覆盖、门控不靠 X——来源于真实的仿真/综合行为差，写进方法论；
- **16 项回归体系**：新增功能的验证模板（迷你几何 + 位精确金标准 + 场景覆盖状态机）；
- **一处诚实标注的待办**：渲染器 v2 补齐后恢复整帧比对，上板前完成。

**五步全部完成**

至此视觉方向软件层收官：看（Step1）- 认（Step2）- 动（Step3）- 显（Step4）- 全（Step5），16 项回归全绿，接口文档 v1.1.0，五册教程 + 全部迭代记录可交付答辩。

# 结语

## 总结与下一步

### Step5 学到了什么

- 官方核的复用姿势（端口兼容、粒度对齐、并行常开）
- 帧边界锁存切换：延迟 1 帧换画面无缝
- Q6 定点的亮度/对比度流水线（居中-缩放-平移-clamp）
- 1bit/像素帧差检测：块存储、同地址读旧写新、亚门限语义
- 仿真/综合不一致两陷阱（越界写回绕、X 门控失效）与两条铁律
- 三行 OSD 与声光反馈的集成

### 下一步做什么

- **上板**：五步模块全部挂进官方工程，TD 综合到 PH1P35，板上标定阈值与增益（IF-SERVO-PWM / IF-OSD 的现场流程）
- **遗留项**：金标准渲染器升级到 v2（三行文本 + 跑马灯），恢复 tb_osd_track 的整帧逐帧比对（详见 Part II" 诚实的待办"）
- **答辩**：五册教程 + 全部迭代记录就是最好的过程性证据